

L12c: Deep Q-Learning

Q-networks and the DQN algorithm

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How can an agent learn action values when its states are too many for a table?

Today's concepts

- Explain the limits of a Q-table: the number of states grows exponentially with the number of state variables, and a table learns nothing about a state from the states around it.
- Write the DQN loss and its gradient step: deep Q-learning fits a Q-network to targets built from its own estimates, and tabular Q-learning is a special case.
- Describe the replay buffer and the target network: each one addresses a problem of Q-learning with a network.

Lecture: Deep Q-Learning

Lecture and Example

Lecture: Deep Q-Learning

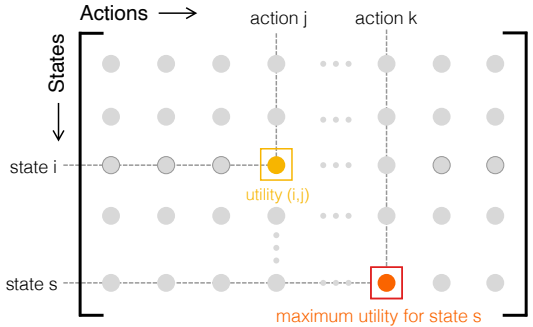
In this lecture, we replace the Q-table with a function that computes the action values from a few numbers that describe the state. This idea is the basis of deep Q-learning and of its best-known method, the deep Q-network (DQN). We start with the simplest such function, a linear one.

Example: Learn a dosing policy without a grid using a linear DQN

In this example, an agent learns to dose the patient of the dosing lab without rounding the tumor burden and the toxicity to grid levels. Its linear Q-network has 148 weights, where the dosing lab's Q-table has 6,724 entries.

The Q-Table

Tabular Q-learning stores one action value for every state-action pair.



Entry (i, j) holds the value (utility) of taking action j in state i. The policy reads each row and picks the column with the largest entry.

The Curse of Dimensionality

The Q-table has three problems:

- The table grows exponentially: a state with m variables, each rounded to L levels, has L^m states. The dosing-lab patient has $41^2 = 1681$ states, but ten variables would give about 1.3×10^{16}.
- The table does not generalize: each entry learns only from visits to its own state-action pair. Two states with almost the same tumor burden share nothing.
- Rounding changes the problem: the dosing lab's policy gives no dose at a tumor burden of 0.109, because its grid rounds 0.109 to 0.100, a level that counts as a cure.

Richard Bellman called this exponential growth the curse of dimensionality.

Q-Networks

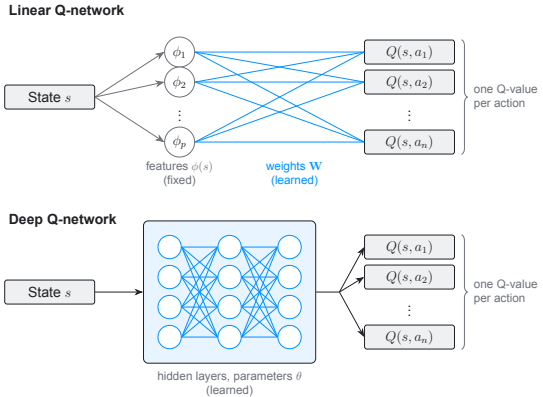
Instead, we replace the table with a function that computes the action values from a few numbers that describe the state.

A Q-network is a function $Q(s, a; \theta)$ with adjustable parameters θ. It takes a state s as its input and returns one Q-value for each of the n actions in the action set $\mathcal{A} = \{a_1, \ldots, a_n\}$. The greedy policy picks the action with the largest entry:

$$\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a; \theta)$$

Learning now means adjusting θ instead of the entries of a table. The number of parameters is set by the network, not by the number of states.

Two Q-Networks



In the linear network, the p features $\phi_1(s), \ldots, \phi_p(s)$ are fixed, and only the weights (blue) are learned. In the deep network, the hidden layers are learned as well.

A Linear Q-Network

A linear Q-network describes the state s by a feature vector $\phi(s) \in \mathbb{R}^p$, a list of p numbers computed from the state. The Q-value of action a, a weighted sum of the features with weight vector $\mathbf{w}_a \in \mathbb{R}^p$, is given by:

$$Q(s, a; \mathbf{W}) = \mathbf{w}_a^\top \phi(s) = \sum_{j=1}^{p} w_{a,j}\, \phi_j(s)$$

where $w_{a,j}$ is the weight of action a on feature $\phi_j(s)$. The weight matrix $\mathbf{W}$ has one row $\mathbf{w}_a^\top$ per action, which gives $n \cdot p$ weights in all.

Two Choices of Features

In the example, the state is $s = (B, Z)$, the tumor burden and the toxicity.

Radial basis functions with centers $\mathbf{c}_j$ and width σ:

$$\phi_j(s) = \exp\left(-\frac{\|s - \mathbf{c}_j\|_2^2}{2\sigma^2}\right)$$

where $\|s - \mathbf{c}_j\|_2$ is the distance from s to $\mathbf{c}_j$. Nearby states get similar Q-values. What the agent learns in one state carries over to its neighbors.

One-hot: number the states, and put a one in entry s and zeros everywhere else:

$$Q(s, a; \mathbf{W}) = w_{a,s}$$

The weight matrix $\mathbf{W}$ is the transpose of the Q-table. A Q-table is a linear network.

A Deep Q-Network

A neural network is a function built from layers. A deep Q-network replaces the fixed features with the hidden layers of a neural network, the layers between its input and its output:

- Each hidden layer is a column of units. Each unit takes a weighted sum x of its inputs and passes it through a fixed nonlinear function, such as $\max(0, x)$. The hidden layers learn their own features from the data.
- The output layer takes weighted sums only, which lets a Q-value be negative.
- Mnih et al. (2015) trained a deep Q-network to play Atari video games from the pixels on the screen.

The rest of this lecture holds for any Q-network with parameters θ, the weights $\mathbf{W}$ in the linear case.

The DQN Loss

Q-learning moves the entry $Q(s, a)$ toward the target $r + \gamma \max_{a'} Q(s', a')$, where r is the reward, s' is the next state, and γ is the discount factor. A Q-network has no entry to move:

- Deep Q-learning adjusts θ to shrink the squared gap between the network's prediction and the target. This fit is a regression with the targets as labels.
- The targets are called bootstrap targets, because they are built from the network's own estimates.
- The training data are transitions $(s_i, a_i, r_i, s'_i, d_i)$: a state, the action taken, the reward, the next state, and a done flag d_i. The flag is one if s'_i is absorbing, a state that ends the episode for good, and zero otherwise.

In the example, a cure and a toxic death are absorbing. An episode that stops at a step limit keeps $d_i = 0$ on its last transition.

DQN Bellman Loss

The target y_i of transition i is the value that the prediction $Q(s_i, a_i; \theta)$ should match:

$$y_i = r_i + \gamma\,(1 - d_i)\max_{a' \in \mathcal{A}} Q(s_i', a'; \theta^-)$$

where θ^- are the parameters of the target network, a delayed copy of the Q-network. The factor $(1 - d_i)$ drops the future value when s_i' is absorbing.

On a mini-batch of N_{batch} past transitions, the loss is half the mean of the squared prediction errors $y_i - Q(s_i, a_i; \theta)$:

$$\mathcal{L}(\theta) = \frac{1}{2N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \left(y_i - Q(s_i, a_i; \theta)\right)^2$$

The prediction uses the action a_i that was actually taken. The target uses the best next action under the target network.

The Gradient Step of a Linear Network

One step of gradient descent with learning rate α is $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$, where the gradient $\nabla_\theta \mathcal{L}$ is the vector of derivatives of the loss and α sets the size of the step.

For the linear network, the derivative of the prediction $\mathbf{w}_{a_i}^\top \phi(s_i)$ with respect to $\mathbf{w}_{a_i}$ is the feature vector $\phi(s_i)$. The target y_i comes from the target network and does not depend on $\mathbf{W}$. One step updates the row of each action in the mini-batch:

$$\mathbf{w}_a \leftarrow \mathbf{w}_a + \frac{\alpha}{N_{\text{batch}}} \sum_{i \,:\, a_i = a} \left(y_i - \mathbf{w}_a^\top \phi(s_i)\right) \phi(s_i)$$

A check against the table: with one-hot features, a mini-batch that holds only the transition just taken, and $\theta^- = \theta$ at every step, this update is the tabular Q-learning update.

Two Stabilizers

Tabular Q-learning converges when every non-absorbing state-action pair is visited infinitely often, and the learning rates of each pair add up to infinity while their squares add up to a finite number. Deep Q-learning can instead diverge, with values that grow without bound (Sutton and Barto, 2018). It adds two stabilizers:

- The replay buffer: many gradient steps on consecutive transitions fit one region of states and spoil the values elsewhere. The buffer $\mathcal{D}$ keeps the last M transitions, and each training step uses a random mini-batch of N_{batch} of them.
- The target network: if the Q-network computed its own targets, each gradient step would also move them. A copy of the Q-network, $Q(s, a; \theta^-)$, computes the targets instead, and every C steps the algorithm copies $\theta^- \leftarrow \theta$.

Between copies, training is close to a regression on random examples with fixed labels.

DQN Algorithm: Initialize

Initialize: Given a Q-network $Q(s, a; \theta)$, set θ to zeros for a linear network or to small random values for a deep network, and copy the parameters to the target network, $\theta^{-} \leftarrow \theta$. Start with an empty replay buffer $\mathcal{D}$ and a total step counter $\tau \leftarrow 0$.

Hyperparameters:

- the discount factor $0 \leq \gamma < 1$ and the learning rate $\alpha > 0$,
- the exploration probability ϵ of each episode,
- the number of episodes N and the maximum number of steps per episode $t_{\max}$,
- the buffer size M and the warm-up size N_{warm}, the number of transitions the buffer must hold before training begins,
- the mini-batch size N_{batch} and the target-network copy interval C.

DQN Algorithm

For each of the N episodes **do**:

1. Choose a starting state s that is not absorbing, and set the step counter $t \leftarrow 0$.
2. While s is not absorbing and $t < t_{\max}$ **do**:
   - 2.1 Choose the action a: draw a random number ρ uniformly from $[0, 1]$.
     - If $\rho \leq \epsilon$ **then** choose $a \in \mathcal{A}$ uniformly at random (exploration).
     - Otherwise, choose the greedy action $a \leftarrow \arg\max_{a' \in \mathcal{A}} Q(s, a'; \theta)$ (exploitation).
   - 2.2 Take action a, and observe the reward r, the next state s', and the done flag d from the *world*, where $d = 1$ if s' is absorbing and $d = 0$ otherwise.
   - 2.3 Store the transition (s, a, r, s', d) in the replay buffer $\mathcal{D}$, and remove the oldest transition if $\mathcal{D}$ holds more than M.
   - 2.4 Update the counters, $t \leftarrow t + 1$ and $\tau \leftarrow \tau + 1$.
   - 2.5 If $\mathcal{D}$ holds at least N_{warm} transitions **then** take one training step.
   - 2.6 If τ is a multiple of C **then** copy the Q-network to the target network, $\theta^- \leftarrow \theta$.
   - 2.7 Update the state, $s \leftarrow s'$.

Extract Policy: For any state s, compute $\pi(s) \leftarrow \arg\max_{a \in \mathcal{A}} Q(s, a; \theta)$.

DQN Algorithm: The Training Step

Step 2.5 of the loop takes one training step:

1. Draw a mini-batch of N_{batch} transitions $(s_i, a_i, r_i, s'_i, d_i)$, $i = 1, \ldots, N_{\text{batch}}$, uniformly at random with replacement from $\mathcal{D}$.
2. Compute the target of each transition with the *target network*, $y_i \leftarrow r_i + \gamma\,(1 - d_i)\,\max_{a' \in \mathcal{A}} Q(s'_i, a'; \theta^-)$.
3. Compute the loss on the actions that were actually taken:

$$\mathcal{L}(\theta) = \frac{1}{2N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \left(y_i - Q(s_i, a_i; \theta)\right)^2$$

4. Take a *single* gradient descent step on the loss, $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$.
   - *Why only a single step?* Many steps on one small mini-batch can overfit its few transitions and undo what the network learned from the rest of the buffer.

Notes on the Algorithm

- The policy gives an action for any state, including states the agent never visited.
- *Why random starting weights for a deep network?* If all the weights start equal, the units of a hidden layer compute the same feature, get the same update, and stay equal. A linear network has fixed features, and zeros work.
- In the example, a linear network with 36 basis functions and a constant feature trains for 3,000 episodes in about one second. The same algorithm trains a deep network, which needs a package for neural networks and much more time.

Example: A Linear DQN for Adaptive Dosing

In this example, an agent learns to dose the patient of the dosing lab without a grid:

1. Build the patient model, with the tumor burden and the toxicity kept as continuous numbers.
2. Train a linear DQN with 36 basis functions and a constant feature, 148 weights in all.
3. Compare its doses with value iteration on the dosing lab's 41-level grid and on a 201-level grid.

From the dosing lab's starting state, the linear DQN cures the patient in 23 cycles and the 201-level policy in 19. The 41-level policy does not cure the patient within 100 cycles.

Example: A linear DQN for adaptive drug dosing

Overestimation

The DQN target uses the largest estimated Q-value of the next state:

- Every estimate has some error, and the largest one is often an estimate whose error is positive.
- The target then tends to be too high on average, even when each estimate is right on average. This upward bias is called overestimation.
- Overestimation spreads back to the values of earlier states, and it can make a worse action look like the best one.

Double DQN

The maximum uses the same estimates to choose the next action and to score it. Double DQN (van Hasselt, Guez, and Silver, 2016) chooses with the Q-network and scores with the target network. Its target is given by:

$$y_i = r_i + \gamma\,(1 - d_i)\,Q\Big(s'_i,\ \arg\max_{a' \in \mathcal{A}} Q(s'_i, a'; \theta);\ \theta^-\Big)$$

The target network is a recent copy of the Q-network, and the two share part of their errors. Double DQN can reduce overestimation, but it does not always remove it.

Lab: Overestimation and Double DQN

In the lab, we'll measure overestimation and watch it make Q-learning and DQN prefer the worse action on a small problem with two states:

- Measure how far the largest of several noisy estimates lands above the true largest value.
- Compare Q-learning with Double Q-learning, which keeps two Q-tables that score each other's choices.
- Test Double DQN, and measure how often it still prefers the worse action and how that depends on how often we copy the target network.

Lab: Overestimation and Double DQN

Summary

We replaced the Q-table with a Q-network and built deep Q-learning around it.

Key takeaways

- **Q-networks.** A Q-network computes every action value from features of the state, with a fixed number of parameters.
- **The DQN loss.** Deep Q-learning fits the network by regression on bootstrap targets from the target network.
- **Stabilizers.** Random mini-batches from the replay buffer and a fixed target network make training steadier.

A linear network with well-chosen features is often enough for a problem with a few state variables. A deep network learns its own features, which lets the same algorithm work from raw inputs such as images.